RAG (Retrieval-Augmented Generation) 전처리 방법론

> **2025년 최신 트렌드 기반**
>
> 참고: Firecrawl, Anthropic, Jina AI

---

1. RAG 개요

1.1 RAG란?

RAG(Retrieval-Augmented Generation)는 LLM의 생성 능력과 외부 지식 검색을 결합한 아키텍처입니다.

```
[사용자 질문] → [검색(Retrieval)] → [관련 문서 청크] → [LLM 생성(Generation)] → [답변]
```

1.2 RAG 파이프라인 구성요소

```
┌──────────────────────────────────────────────────────────────────┐
│                        RAG Pipeline                              │
├──────────────────────────────────────────────────────────────────┤
│  1. Ingestion (수집)                                             │
│     └─ 문서 파싱 → 전처리 → 청킹 → 임베딩 → 벡터 DB 저장         │
│                                                                  │
│  2. Retrieval (검색)                                             │
│     └─ 질문 임베딩 → 유사도 검색 → Top-K 청크 반환               │
│                                                                  │
│  3. Generation (생성)                                            │
│     └─ 프롬프트 구성 → LLM 호출 → 답변 생성                      │
└──────────────────────────────────────────────────────────────────┘
```

---

2. 청킹(Chunking) 전략

> **"Your goal is not to chunk for chunking sake, our goal is to get our data in a format where it can be retrieved for value later."** - Greg Kamradt

2.0 Greg Kamradt의 5 Levels of Text Splitting

> 참고: GitHub Tutorial, Medium 정리

```
┌──────────────────────────────────────────────────────────────────┐
│              5 Levels of Text Splitting                          │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
```

```
│  Level 1: Character Splitting                                    │
│  └ 단순 문자 수 기준 분할 (가장 기초적)                          │
│                                                                  │
│  Level 2: Recursive Character Text Splitter                      │
│  └ 구분자 계층 (\n\n → \n → . → " ") 기반 분할                   │
│                                                                  │
│  Level 3: Document-Specific Splitting                            │
│  └ 문서 유형별 구조 인식 (Markdown, Code, PDF)                   │
│                                                                  │
│  Level 4: Semantic Splitting (Embeddings)                        │
│  └ 문장 임베딩 유사도 기반 의미 단위 분할                        │
│                                                                  │
│  Level 5: Agentic Splitting                                      │
│  └ LLM/Agent가 문맥 기반으로 청크 결정                           │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Level 4 - Semantic Chunking 원리 (Greg's Walk Method):

```python
# 문장별 임베딩 생성 후 순차 비교
embeddings = [embed(sent) for sent in sentences]

chunks = []
current_chunk = [sentences[0]]

for i in range(1, len(sentences)):
    # 인접 문장 간 유사도 측정
    similarity = cosine_similarity(embeddings[i-1], embeddings[i])

    if similarity < threshold:  # 유사도가 낮으면 = 의미적 분리점
        chunks.append(current_chunk)
        current_chunk = [sentences[i]]
    else:
        current_chunk.append(sentences[i])

# Break Point = 임베딩 거리가 큰 지점
```

Level 5 - Agentic Splitting (Propositions 기반):

```python
# LLM이 텍스트를 독립적인 명제(Proposition)로 분해
PROPOSITION_PROMPT = """
다음 텍스트를 독립적으로 이해 가능한 명제들로 분해하세요.
각 명제는 문맥 없이도 의미가 명확해야 합니다.

텍스트: {text}
"""

# 예시:
# 입력: "삼성전자가 서울에 본사를 둔 회사로, 이재용이 회장이다."
```

```
# 출력:
# - "삼성전자는 회사이다."
# - "삼성전자의 본사는 서울에 있다."
# - "이재용은 삼성전자의 회장이다."
```

2.1 기본 청킹 방식 (Level 1-2)

Fixed-Size Chunking (고정 크기)

```
# 단순하지만 의미 경계 무시
chunk_size = 1000  # 문자 또는 토큰
overlap = 150      # 오버랩

chunks = []
for i in range(0, len(text), chunk_size - overlap):
    chunks.append(text[i:i + chunk_size])
```

장점: 구현 간단, 예측 가능한 크기 **단점:** 의미 단위 무시, 문맥 손실

Sentence-Based Chunking (문장 기반)

```
import nltk
sentences = nltk.sent_tokenize(text)
# 문장 단위로 그룹화하여 청크 생성
```

Recursive Character Text Splitter

```
# LangChain 스타일
separators = ["\n\n", "\n", ". ", " ", ""]
# 계층적으로 분할 시도
```

2.2 2025 최신 청킹 전략

Late Chunking (Jina AI)

개념: 전체 문서를 먼저 임베딩한 후 청킹 → 문맥 손실 방지

```
[기존 방식]
문서 → 청킹 → 각 청크 임베딩 (문맥 손실!)

[Late Chunking]
문서 → 전체 임베딩 (Transformer) → 청킹 (Mean Pooling)
```

구현 원리:

```
# 1. 전체 문서를 Long-context 모델로 토큰화
tokens = tokenizer(full_document)

# 2. Transformer layer 통과 (전체 문맥 반영)
hidden_states = transformer(tokens)

# 3. 청크 경계에서 Mean Pooling
chunk_embeddings = []
for start, end in chunk_boundaries:
    chunk_emb = mean_pool(hidden_states[start:end])
    chunk_embeddings.append(chunk_emb)
```

지원 모델:

- jina-embeddings-v3 (8K context)
- nomic-embed-text-v1 (8K context)
- GTE-large (8K context)

장점:

- 추가 저장공간 불필요 (임베딩 크기 동일)
- LLM 호출 없이 빠름
- 청크 경계에 강건함

Contextual Retrieval (Anthropic)

개념: 각 청크에 LLM으로 문맥 정보 추가 → 검색 정확도 67% 향상

```
[Before]
청크: "매출이 3% 증가했습니다."
→ 어떤 회사? 어느 분기?

[After]
청크: "[ACME Corp 2023 Q2 실적 보고서] 매출이 3% 증가했습니다."
→ 문맥 명확!
```

구현:

```
CONTEXT_PROMPT = """
<document>
{WHOLE_DOCUMENT}
</document>
```

```
Here is the chunk we want to situate within the whole document:
<chunk>
{CHUNK_CONTENT}
</chunk>

Please give a short succinct context to situate this chunk within
the overall document for the purposes of improving search retrieval
of the chunk. Answer only with the succinct context and nothing else.
"""

def add_context(chunk, document):
    context = llm.generate(CONTEXT_PROMPT.format(
        WHOLE_DOCUMENT=document,
        CHUNK_CONTENT=chunk
    ))
    return f"{context}\n\n{chunk}"  # 50-100 토큰 추가
```

비용 최적화 (Prompt Caching):

```
# 문서를 캐시에 한 번만 로드
cached_doc = prompt_cache.store(document)

# 각 청크 처리 시 캐시 참조
for chunk in chunks:
    context = llm.generate(
        cached_document=cached_doc,
        chunk=chunk
    )
```

예상 비용: $1.02 / 1M 문서 토큰 (캐싱 사용 시)

Late Chunking vs Contextual Retrieval 비교

> 참고: Math Behind RAG's Context Problem

구분	Late Chunking	Contextual Retrieval
방식	전체 임베딩 → 청킹	청크 + LLM 문맥 추가
문맥 보존	Transformer 내부	명시적 텍스트 추가
비용	낮음 (임베딩만)	높음 (LLM 호출)
속도	빠름	느림
저장공간	동일	50-100 토큰 증가
문맥 윈도우	임베딩 모델 제한 (8K)	LLM 제한 (128K+)
유사도 점수	청크 간 유사	청크 간 유사

선택 가이드:

```
IF 비용 민감 AND 문서 < 8K 토큰:
    → Late Chunking
ELIF 정확도 최우선 AND 비용 여유:
    → Contextual Retrieval
ELIF 매우 긴 문서 (>8K):
    → Contextual Retrieval (LLM 윈도우 활용)
ELSE:
    → Hybrid (Late Chunking + Reranker)
```

Agentic Chunking (Level 5)

개념: AI 에이전트가 문서 특성에 따라 최적 전략 자동 선택

```python
class AgenticChunker:
    def analyze_document(self, doc):
        """문서 특성 분석"""
        return {
            "type": "technical_report",  # 문서 유형
            "density": "high",            # 정보 밀도
            "structure": "hierarchical",  # 구조
            "avg_section_length": 500
        }

    def select_strategy(self, analysis):
        """전략 선택"""
        if analysis["density"] == "high":
            return "small_chunks"  # 250 토큰
        elif analysis["structure"] == "hierarchical":
            return "header_based"  # 헤딩 기준
        else:
            return "semantic"      # 의미 기반
```

Semantic Chunking (SDPM)

개념: 문장 임베딩 유사도 기반으로 의미 단위 분할

```python
from chonkie import SDPMChunker

chunker = SDPMChunker(
    embedding_model="sentence-transformers/all-MiniLM-L6-v2",
    threshold=0.5,  # 유사도 임계값
    min_chunk_size=100,
    max_chunk_size=500
)
```

```
chunks = chunker.chunk(text)
```

Semantic Double-Pass Merge (SDPM):

1. 1차: 문장별 임베딩 생성
2. 2차: 유사도 기반 문장 그룹화
3. 3차: 그룹 병합 및 경계 최적화

---

3. Document-Aware Chunking

3.1 구조 인식 분할

문서의 자연스러운 경계를 존중:

```
# Markdown
separators = ["# ", "## ", "### ", "\n\n", "\n"]

# HTML
separators = ["</h1>", "</h2>", "</section>", "</p>"]

# PDF (Docling 사용)
from docling import DocumentConverter
converter = DocumentConverter()
result = converter.convert("document.pdf")
# → 헤딩, 단락, 표 구조 보존
```

3.2 표/차트 보존

핵심 원칙: 표와 차트는 절대 분할하지 않음

```
def chunk_with_tables(document):
    chunks = []

    for element in document.elements:
        if element.type == "TABLE":
            # 표는 독립 청크로 보존
            chunks.append({
                "type": "table",
                "content": element.content,
                "metadata": element.headers
            })
        else:
            # 일반 텍스트는 청킹
            text_chunks = chunker.chunk(element.content)
            chunks.extend(text_chunks)

    return chunks
```

---

4. Hybrid Retrieval

4.1 Dense + Sparse 검색

```
┌──────────────────────────────────────────────┐
│              Hybrid Retrieval                 │
├──────────────────────────────────────────────┤
│                                              │
│  Query: "ACME 2023 매출"                     │
│          │                                   │
│     ┌────┴────┐                             │
│     ▼         ▼                             │
│  [Dense]    [Sparse]                         │
│  Embedding   BM25                            │
│     │         │                              │
│     ▼         ▼                              │
│  Top-K      Top-K                            │
│     │         │                              │
│     └────┬────┘                              │
│          ▼                                   │
│    [Reciprocal Rank Fusion]                  │
│          │                                   │
│          ▼                                   │
│      [Reranker]                              │
│          │                                   │
│          ▼                                   │
│     Final Results                            │
└──────────────────────────────────────────────┘
```

4.2 구현 예시

```python
from rank_bm25 import BM25Okapi
import numpy as np

class HybridRetriever:
    def __init__(self, chunks, embeddings, alpha=0.5):
        self.chunks = chunks
        self.embeddings = embeddings
        self.alpha = alpha

        # BM25 인덱스
        tokenized = [c.split() for c in chunks]
        self.bm25 = BM25Okapi(tokenized)

    def search(self, query, top_k=20):
        # Dense search
        query_emb = embed(query)
```

```
        dense_scores = cosine_similarity(query_emb, self.embeddings)

        # Sparse search (BM25)
        sparse_scores = self.bm25.get_scores(query.split())

        # Normalize and combine
        dense_norm = (dense_scores - dense_scores.min()) / (dense_scores.max() -
dense_scores.min())
        sparse_norm = (sparse_scores - sparse_scores.min()) / (sparse_scores.max()
- sparse_scores.min())

        combined = self.alpha * dense_norm + (1 - self.alpha) * sparse_norm

        top_indices = np.argsort(combined)[-top_k:][::-1]
        return [self.chunks[i] for i in top_indices]
```

4.3 Reranking

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

def rerank(query, chunks, top_k=5):
    pairs = [[query, chunk] for chunk in chunks]
    scores = reranker.predict(pairs)

    ranked = sorted(zip(chunks, scores), key=lambda x: x[1], reverse=True)
    return [chunk for chunk, score in ranked[:top_k]]
```

---

5. 권장 파라미터 (2025)

파라미터	권장값	출처
청크 크기	250 토큰 (~1000자)	Firecrawl
오버랩	15% (약 37 토큰)	NVIDIA
Top-K 검색	20 청크	Anthropic
Reranker 후	5-10 청크	Best Practice

---

6. 평가 메트릭

6.1 검색 품질

```
def evaluate_retrieval(queries, ground_truth, retrieved):
    metrics = {
        "recall@k": [],
```

```
        "precision@k": [],
        "mrr": [],
        "ndcg": []
    }

    for q, gt, ret in zip(queries, ground_truth, retrieved):
        # Recall@K
        hits = len(set(gt) & set(ret))
        metrics["recall@k"].append(hits / len(gt))

        # MRR
        for i, doc in enumerate(ret):
            if doc in gt:
                metrics["mrr"].append(1 / (i + 1))
                break
        else:
            metrics["mrr"].append(0)

    return {k: np.mean(v) for k, v in metrics.items()}
```

6.2 RAG 품질

- **Faithfulness:** 답변이 검색된 문서에 충실한가?
- **Answer Relevancy:** 답변이 질문에 관련있는가?
- **Context Precision:** 검색된 문맥이 정확한가?
- **Context Recall:** 필요한 정보가 모두 검색되었는가?

---

7. 도구 및 라이브러리

도구	용도	설치
Chonkie	청킹 라이브러리	`pip install chonkie[all]`
LangChain	RAG 파이프라인	`pip install langchain`
LlamaIndex	문서 인덱싱	`pip install llama-index`
ChromaDB	벡터 DB	`pip install chromadb`
Docling	문서 파싱	`pip install docling`

---

8. 체크리스트

전처리 체크리스트

- ☐ 문서 파싱 (PDF, DOCX, HTML 등)
- ☐ 텍스트 정제 (노이즈 제거, 정규화)
- ☐ 구조 추출 (헤딩, 표, 이미지)
- ☐ 청킹 전략 선택
- ☐ 문맥 추가 (Contextual Retrieval)

- ☐ 임베딩 생성
- ☐ 벡터 DB 저장
- ☐ BM25 인덱스 생성 (하이브리드)

검색 최적화 체크리스트

- ☐ 하이브리드 검색 설정
- ☐ Reranker 적용
- ☐ Top-K 최적화
- ☐ 평가 메트릭 측정

---

9. 임베딩 모델 상세 비교 (2025)

9.1 주요 임베딩 모델

모델	차원	컨텍스트	MTEB 점수	용도	비용
text-embedding-3-large	3072	8K	64.6	범용, 프로덕션	$0.13/1M 토큰
text-embedding-3-small	1536	8K	62.3	비용 효율	$0.02/1M 토큰
jina-embeddings-v3	1024	8K	65.5	Late Chunking	오픈소스
nomic-embed-text-v1.5	768	8K	62.3	오픈소스	무료
voyage-3	1024	32K	67.3	장문서	$0.06/1M 토큰
voyage-3-lite	512	32K	63.5	경량	$0.02/1M 토큰
Cohere embed-v3	1024	512	64.5	다국어	$0.10/1M 토큰
bge-m3	1024	8K	66.4	다국어, Sparse+Dense	오픈소스
multilingual-e5-large	1024	512	61.5	다국어	오픈소스

9.2 한국어 특화 모델

모델	차원	특징	KorSTS 점수
ko-sroberta-multitask	768	한국어 특화	84.2
KoSimCSE-roberta	768	대조학습	83.5
paraphrase-multilingual-MiniLM	384	경량 다국어	78.3
bge-m3	1024	다국어 + Sparse	82.1

9.3 모델 선택 가이드

```
def select_embedding_model(requirements: dict) -> str:
    """
    요구사항에 따른 임베딩 모델 선택
```

```
    Args:
        requirements: {
            "language": "ko" | "en" | "multi",
            "budget": "low" | "medium" | "high",
            "accuracy": "low" | "medium" | "high",
            "document_length": "short" | "medium" | "long",
            "use_case": "rag" | "search" | "clustering"
        }
    """
    lang = requirements.get("language", "en")
    budget = requirements.get("budget", "medium")
    accuracy = requirements.get("accuracy", "medium")
    doc_length = requirements.get("document_length", "medium")

    # 한국어 우선
    if lang == "ko":
        if accuracy == "high":
            return "bge-m3"  # 다국어 최고 성능
        elif budget == "low":
            return "paraphrase-multilingual-MiniLM-L12-v2"
        else:
            return "ko-sroberta-multitask"

    # 영어
    if lang == "en":
        if doc_length == "long":
            return "voyage-3"  # 32K 컨텍스트
        elif accuracy == "high" and budget == "high":
            return "text-embedding-3-large"
        elif budget == "low":
            return "nomic-embed-text-v1.5"
        else:
            return "text-embedding-3-small"

    # 다국어
    if lang == "multi":
        return "bge-m3"

    return "text-embedding-3-small"  # 기본값
```

9.4 임베딩 생성 코드

```
from typing import List
import numpy as np

class EmbeddingGenerator:
    """통합 임베딩 생성기"""

    def __init__(self, model_name: str):
        self.model_name = model_name
        self._load_model()
```

```python
    def _load_model(self):
        if "text-embedding" in self.model_name:
            # OpenAI
            from openai import OpenAI
            self.client = OpenAI()
            self.embed_fn = self._openai_embed
        elif "voyage" in self.model_name:
            # Voyage AI
            import voyageai
            self.client = voyageai.Client()
            self.embed_fn = self._voyage_embed
        elif "bge" in self.model_name or "e5" in self.model_name:
            # HuggingFace
            from sentence_transformers import SentenceTransformer
            self.model = SentenceTransformer(self.model_name)
            self.embed_fn = self._hf_embed
        elif "jina" in self.model_name:
            # Jina AI
            from sentence_transformers import SentenceTransformer
            self.model = SentenceTransformer(
                self.model_name,
                trust_remote_code=True
            )
            self.embed_fn = self._hf_embed

    def embed(self, texts: List[str], batch_size: int = 32) -> np.ndarray:
        """텍스트 임베딩 생성"""
        embeddings = []
        for i in range(0, len(texts), batch_size):
            batch = texts[i:i + batch_size]
            batch_emb = self.embed_fn(batch)
            embeddings.extend(batch_emb)
        return np.array(embeddings)

    def _openai_embed(self, texts: List[str]) -> List[List[float]]:
        response = self.client.embeddings.create(
            model=self.model_name,
            input=texts
        )
        return [e.embedding for e in response.data]

    def _voyage_embed(self, texts: List[str]) -> List[List[float]]:
        response = self.client.embed(
            texts,
            model=self.model_name,
            input_type="document"
        )
        return response.embeddings

    def _hf_embed(self, texts: List[str]) -> List[List[float]]:
        return self.model.encode(texts).tolist()
```

```
# 사용 예시
generator = EmbeddingGenerator("text-embedding-3-small")
embeddings = generator.embed(["안녕하세요", "반갑습니다"])
print(f"Shape: {embeddings.shape}")  # (2, 1536)
```

---

10. 벡터 데이터베이스 통합

10.1 벡터 DB 비교

DB	타입	스케일	특징	적합 환경
ChromaDB	오픈소스	소규모	간단, 로컬	프로토타입, 개발
Pinecone	매니지드	대규모	서버리스, 고가용성	프로덕션
Weaviate	오픈소스	중대규모	하이브리드 검색	자체 호스팅
Qdrant	오픈소스	중대규모	Rust 기반, 고성능	자체 호스팅
Milvus	오픈소스	대규모	분산, GPU 지원	엔터프라이즈
pgvector	확장	중규모	PostgreSQL 통합	기존 PG 사용자
FAISS	라이브러리	대규모	인메모리, 빠름	연구, 배치

10.2 ChromaDB 통합

```
import chromadb
from chromadb.config import Settings

class ChromaDBStore:
    """ChromaDB 벡터 스토어"""

    def __init__(self,
                 collection_name: str,
                 persist_directory: str = "./chroma_db"):
        self.client = chromadb.Client(Settings(
            chroma_db_impl="duckdb+parquet",
            persist_directory=persist_directory,
            anonymized_telemetry=False
        ))
        self.collection = self.client.get_or_create_collection(
            name=collection_name,
            metadata={"hnsw:space": "cosine"}
        )

    def add(self,
            ids: list[str],
            embeddings: list[list[float]],
            documents: list[str],
            metadatas: list[dict] = None):
```

```
        """문서 추가"""
        self.collection.add(
            ids=ids,
            embeddings=embeddings,
            documents=documents,
            metadatas=metadatas
        )

    def search(self,
               query_embedding: list[float],
               top_k: int = 10,
               where: dict = None) -> dict:
        """유사도 검색"""
        results = self.collection.query(
            query_embeddings=[query_embedding],
            n_results=top_k,
            where=where,
            include=["documents", "metadatas", "distances"]
        )
        return results

    def delete(self, ids: list[str]):
        """문서 삭제"""
        self.collection.delete(ids=ids)


# 사용 예시
store = ChromaDBStore("my_documents")

# 문서 추가
store.add(
    ids=["doc1", "doc2"],
    embeddings=[[0.1, 0.2, ...], [0.3, 0.4, ...]],
    documents=["첫 번째 문서", "두 번째 문서"],
    metadatas=[{"source": "file1.pdf"}, {"source": "file2.pdf"}]
)

# 검색
results = store.search(
    query_embedding=[0.15, 0.25, ...],
    top_k=5,
    where={"source": "file1.pdf"}
)
```

10.3 Pinecone 통합

```
from pinecone import Pinecone, ServerlessSpec

class PineconeStore:
    """Pinecone 벡터 스토어"""
```

```python
    def __init__(self,
                 api_key: str,
                 index_name: str,
                 dimension: int = 1536):
        self.pc = Pinecone(api_key=api_key)

        # 인덱스 생성 (없으면)
        if index_name not in self.pc.list_indexes().names():
            self.pc.create_index(
                name=index_name,
                dimension=dimension,
                metric="cosine",
                spec=ServerlessSpec(
                    cloud="aws",
                    region="us-east-1"
                )
            )

        self.index = self.pc.Index(index_name)

    def upsert(self,
               vectors: list[dict],
               namespace: str = "default"):
        """
        벡터 업서트

        vectors: [
            {"id": "doc1", "values": [...], "metadata": {...}},
            ...
        ]
        """
        # 배치 처리 (최대 100개)
        batch_size = 100
        for i in range(0, len(vectors), batch_size):
            batch = vectors[i:i + batch_size]
            self.index.upsert(vectors=batch, namespace=namespace)

    def search(self,
               query_vector: list[float],
               top_k: int = 10,
               namespace: str = "default",
               filter: dict = None) -> dict:
        """유사도 검색"""
        results = self.index.query(
            vector=query_vector,
            top_k=top_k,
            namespace=namespace,
            filter=filter,
            include_metadata=True
        )
        return results

    def delete(self,
               ids: list[str] = None,
```

```
                namespace: str = "default",
                delete_all: bool = False):
        """벡터 삭제"""
        if delete_all:
            self.index.delete(delete_all=True, namespace=namespace)
        else:
            self.index.delete(ids=ids, namespace=namespace)


# 사용 예시
store = PineconeStore(
    api_key="your-api-key",
    index_name="documents",
    dimension=1536
)

# 업서트
store.upsert([
    {
        "id": "doc1",
        "values": [0.1, 0.2, ...],
        "metadata": {
            "source": "report.pdf",
            "page": 1,
            "text": "원본 텍스트..."
        }
    }
])

# 검색
results = store.search(
    query_vector=[0.15, 0.25, ...],
    top_k=10,
    filter={"source": {"$eq": "report.pdf"}}
)
```

10.4 Qdrant 통합

```
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Distance, VectorParams, PointStruct,
    Filter, FieldCondition, MatchValue
)

class QdrantStore:
    """Qdrant 벡터 스토어"""

    def __init__(self,
                 host: str = "localhost",
                 port: int = 6333,
                 collection_name: str = "documents",
```

```
                 dimension: int = 1536):
        self.client = QdrantClient(host=host, port=port)
        self.collection_name = collection_name

        # 컬렉션 생성
        collections = self.client.get_collections().collections
        if collection_name not in [c.name for c in collections]:
            self.client.create_collection(
                collection_name=collection_name,
                vectors_config=VectorParams(
                    size=dimension,
                    distance=Distance.COSINE
                )
            )

    def upsert(self, points: list[PointStruct]):
        """포인트 업서트"""
        self.client.upsert(
            collection_name=self.collection_name,
            points=points
        )

    def search(self,
               query_vector: list[float],
               top_k: int = 10,
               filter_conditions: dict = None) -> list:
        """유사도 검색"""
        query_filter = None
        if filter_conditions:
            query_filter = Filter(
                must=[
                    FieldCondition(
                        key=k,
                        match=MatchValue(value=v)
                    )
                    for k, v in filter_conditions.items()
                ]
            )

        results = self.client.search(
            collection_name=self.collection_name,
            query_vector=query_vector,
            limit=top_k,
            query_filter=query_filter
        )
        return results


# 사용 예시
store = QdrantStore(collection_name="my_docs", dimension=1536)

# 추가
points = [
    PointStruct(
```

```
        id=1,
        vector=[0.1, 0.2, ...],
        payload={"source": "doc.pdf", "text": "내용..."}
    )
]
store.upsert(points)

# 검색
results = store.search(
    query_vector=[0.15, 0.25, ...],
    top_k=5,
    filter_conditions={"source": "doc.pdf"}
)
```

---

11. 전체 RAG 파이프라인 구현

11.1 통합 파이프라인

```
from dataclasses import dataclass
from typing import List, Optional
import hashlib

@dataclass
class RAGConfig:
    """RAG 파이프라인 설정"""
    # 청킹 설정
    chunk_size: int = 1000
    chunk_overlap: int = 150
    chunking_strategy: str = "recursive"  # recursive, semantic, late

    # 임베딩 설정
    embedding_model: str = "text-embedding-3-small"
    embedding_batch_size: int = 32

    # 검색 설정
    top_k: int = 20
    use_hybrid: bool = True
    hybrid_alpha: float = 0.5  # dense 가중치
    use_reranker: bool = True
    rerank_top_k: int = 5

    # 벡터 DB 설정
    vector_db: str = "chromadb"  # chromadb, pinecone, qdrant
    collection_name: str = "documents"


class RAGPipeline:
    """통합 RAG 파이프라인"""

    def __init__(self, config: RAGConfig):
```

```
        self.config = config
        self._init_components()

    def _init_components(self):
        """컴포넌트 초기화"""
        # 청커
        if self.config.chunking_strategy == "recursive":
            from src.chunking import RecursiveChunker
            self.chunker = RecursiveChunker(
                chunk_size=self.config.chunk_size,
                chunk_overlap=self.config.chunk_overlap
            )
        elif self.config.chunking_strategy == "semantic":
            from src.chunking import SemanticChunker
            self.chunker = SemanticChunker(
                max_chunk_size=self.config.chunk_size
            )

        # 임베딩 생성기
        self.embedder = EmbeddingGenerator(self.config.embedding_model)

        # 벡터 스토어
        if self.config.vector_db == "chromadb":
            self.vector_store = ChromaDBStore(self.config.collection_name)
        elif self.config.vector_db == "pinecone":
            self.vector_store = PineconeStore(
                api_key=os.environ["PINECONE_API_KEY"],
                index_name=self.config.collection_name
            )

        # Reranker
        if self.config.use_reranker:
            from sentence_transformers import CrossEncoder
            self.reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

        # BM25 (하이브리드용)
        if self.config.use_hybrid:
            from rank_bm25 import BM25Okapi
            self.bm25 = None  # 문서 추가 시 초기화
            self.bm25_corpus = []

    def ingest(self,
               documents: List[dict],
               show_progress: bool = True) -> dict:
        """
        문서 인제스트

        documents: [
            {"id": "doc1", "content": "...", "metadata": {...}},
            ...
        ]
        """
        from tqdm import tqdm
```

```
        all_chunks = []
        all_ids = []
        all_metadatas = []

        doc_iter = tqdm(documents) if show_progress else documents

        for doc in doc_iter:
            # 청킹
            chunks = self.chunker.chunk(doc["content"])

            for i, chunk in enumerate(chunks):
                chunk_id = self._generate_id(doc["id"], i)
                all_chunks.append(chunk.text if hasattr(chunk, 'text') else chunk)
                all_ids.append(chunk_id)
                all_metadatas.append({
                    **doc.get("metadata", {}),
                    "doc_id": doc["id"],
                    "chunk_index": i
                })

        # 임베딩 생성
        embeddings = self.embedder.embed(
            all_chunks,
            batch_size=self.config.embedding_batch_size
        )

        # 벡터 DB 저장
        self.vector_store.add(
            ids=all_ids,
            embeddings=embeddings.tolist(),
            documents=all_chunks,
            metadatas=all_metadatas
        )

        # BM25 인덱스 업데이트
        if self.config.use_hybrid:
            self.bm25_corpus.extend(all_chunks)
            tokenized = [c.split() for c in self.bm25_corpus]
            self.bm25 = BM25Okapi(tokenized)

        return {
            "total_chunks": len(all_chunks),
            "documents_processed": len(documents)
        }

    def retrieve(self,
                 query: str,
                 top_k: Optional[int] = None,
                 filter: dict = None) -> List[dict]:
        """문서 검색"""
        top_k = top_k or self.config.top_k

        # Dense 검색
        query_embedding = self.embedder.embed([query])[0]
```

```
        dense_results = self.vector_store.search(
            query_embedding=query_embedding.tolist(),
            top_k=top_k * 2 if self.config.use_hybrid else top_k,
            where=filter
        )

        # 하이브리드 검색
        if self.config.use_hybrid and self.bm25:
            # Sparse 검색 (BM25)
            sparse_scores = self.bm25.get_scores(query.split())

            # RRF (Reciprocal Rank Fusion)
            results = self._rrf_fusion(dense_results, sparse_scores, top_k)
        else:
            results = self._format_results(dense_results)

        # Reranking
        if self.config.use_reranker:
            results = self._rerank(query, results, self.config.rerank_top_k)

        return results

    def _generate_id(self, doc_id: str, chunk_index: int) -> str:
        """청크 ID 생성"""
        return hashlib.md5(f"{doc_id}_{chunk_index}".encode()).hexdigest()

    def _rrf_fusion(self,
                    dense_results: dict,
                    sparse_scores: list,
                    top_k: int) -> List[dict]:
        """RRF 결합"""
        import numpy as np

        k = 60  # RRF 상수

        # Dense 순위
        dense_docs = dense_results.get("documents", [[]])[0]
        dense_ranks = {doc: i + 1 for i, doc in enumerate(dense_docs)}

        # Sparse 순위
        sparse_indices = np.argsort(sparse_scores)[::-1]
        sparse_ranks = {self.bm25_corpus[i]: rank + 1
                        for rank, i in enumerate(sparse_indices[:top_k * 2])}

        # RRF 점수 계산
        all_docs = set(dense_ranks.keys()) | set(sparse_ranks.keys())
        rrf_scores = {}

        for doc in all_docs:
            dense_score = 1 / (k + dense_ranks.get(doc, 1000))
            sparse_score = 1 / (k + sparse_ranks.get(doc, 1000))
            rrf_scores[doc] = (
                self.config.hybrid_alpha * dense_score +
                (1 - self.config.hybrid_alpha) * sparse_score
```

```
            )

        # 정렬 및 반환
        sorted_docs = sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)

        results = []
        for doc, score in sorted_docs[:top_k]:
            idx = dense_docs.index(doc) if doc in dense_docs else -1
            metadata = dense_results.get("metadatas", [[]])[0][idx] if idx >= 0
else {}
            results.append({
                "content": doc,
                "score": score,
                "metadata": metadata
            })

        return results

    def _format_results(self, results: dict) -> List[dict]:
        """결과 포맷팅"""
        formatted = []
        documents = results.get("documents", [[]])[0]
        distances = results.get("distances", [[]])[0]
        metadatas = results.get("metadatas", [[]])[0]

        for doc, dist, meta in zip(documents, distances, metadatas):
            formatted.append({
                "content": doc,
                "score": 1 - dist,  # 거리를 유사도로 변환
                "metadata": meta
            })

        return formatted

    def _rerank(self,
                query: str,
                results: List[dict],
                top_k: int) -> List[dict]:
        """Cross-encoder로 재순위"""
        pairs = [[query, r["content"]] for r in results]
        scores = self.reranker.predict(pairs)

        for result, score in zip(results, scores):
            result["rerank_score"] = float(score)

        reranked = sorted(results, key=lambda x: x["rerank_score"], reverse=True)
        return reranked[:top_k]


# 사용 예시
config = RAGConfig(
    chunk_size=500,
    chunking_strategy="recursive",
    embedding_model="text-embedding-3-small",
```

```
    use_hybrid=True,
    use_reranker=True
)

pipeline = RAGPipeline(config)

# 문서 인제스트
pipeline.ingest([
    {"id": "doc1", "content": "문서 내용...", "metadata": {"source":
"report.pdf"}},
    {"id": "doc2", "content": "다른 문서...", "metadata": {"source": "guide.md"}}
])

# 검색
results = pipeline.retrieve("질문 내용")
for r in results:
    print(f"Score: {r['score']:.3f}")
    print(f"Content: {r['content'][:100]}...")
```

11.2 LLM 응답 생성

```
from openai import OpenAI

class RAGGenerator:
    """RAG 응답 생성기"""

    def __init__(self,
                 model: str = "gpt-4o",
                 max_tokens: int = 1024,
                 temperature: float = 0.7):
        self.client = OpenAI()
        self.model = model
        self.max_tokens = max_tokens
        self.temperature = temperature

    def generate(self,
                 query: str,
                 contexts: List[dict],
                 system_prompt: str = None) -> dict:
        """RAG 응답 생성"""

        # 컨텍스트 포맷팅
        context_text = self._format_contexts(contexts)

        # 시스템 프롬프트
        if system_prompt is None:
            system_prompt = """당신은 주어진 문서를 기반으로 질문에 답하는 AI 어시스턴
트입니다.
답변은 반드시 제공된 컨텍스트에 기반해야 합니다.
컨텍스트에 없는 정보는 "제공된 문서에서 해당 정보를 찾을 수 없습니다"라고 답하세요.
답변 시 관련 출처를 인용하세요."""
```

```
        # 프롬프트 구성
        user_prompt = f"""## 참고 문서

{context_text}

## 질문

{query}

## 답변

위 문서를 참고하여 질문에 답하세요:"""

        # LLM 호출
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt}
            ],
            max_tokens=self.max_tokens,
            temperature=self.temperature
        )

        answer = response.choices[0].message.content

        return {
            "answer": answer,
            "contexts": contexts,
            "model": self.model,
            "usage": {
                "prompt_tokens": response.usage.prompt_tokens,
                "completion_tokens": response.usage.completion_tokens,
                "total_tokens": response.usage.total_tokens
            }
        }

    def _format_contexts(self, contexts: List[dict]) -> str:
        """컨텍스트 포맷팅"""
        formatted = []
        for i, ctx in enumerate(contexts, 1):
            source = ctx.get("metadata", {}).get("source", "Unknown")
            content = ctx.get("content", "")
            formatted.append(f"[문서 {i}] (출처: {source})\n{content}")

        return "\n\n---\n\n".join(formatted)


# 전체 사용 예시
pipeline = RAGPipeline(config)
generator = RAGGenerator(model="gpt-4o")

# 검색 + 생성
```

```
query = "프로젝트의 주요 목표는 무엇인가요?"
contexts = pipeline.retrieve(query)
response = generator.generate(query, contexts)

print(f"답변: {response['answer']}")
print(f"토큰 사용량: {response['usage']['total_tokens']}")
```

---

12. 성능 최적화

12.1 청킹 최적화

```
class OptimizedChunker:
    """최적화된 청킹"""

    def __init__(self):
        self.cache = {}

    def chunk_with_cache(self, text: str, doc_id: str) -> list:
        """캐시를 활용한 청킹"""
        cache_key = hashlib.md5(text.encode()).hexdigest()

        if cache_key in self.cache:
            return self.cache[cache_key]

        chunks = self._do_chunk(text)
        self.cache[cache_key] = chunks

        return chunks

    def parallel_chunk(self, documents: list, workers: int = 4) -> list:
        """병렬 청킹"""
        from concurrent.futures import ProcessPoolExecutor

        with ProcessPoolExecutor(max_workers=workers) as executor:
            results = list(executor.map(self._do_chunk, documents))

        return results
```

12.2 임베딩 최적화

```
import asyncio
from typing import List

class AsyncEmbedder:
    """비동기 임베딩 생성"""

    async def embed_async(self, texts: List[str]) -> List[List[float]]:
        """비동기 임베딩"""
```

```
        from openai import AsyncOpenAI

        client = AsyncOpenAI()

        async def embed_batch(batch):
            response = await client.embeddings.create(
                model="text-embedding-3-small",
                input=batch
            )
            return [e.embedding for e in response.data]

        # 배치 처리
        batch_size = 100
        tasks = []
        for i in range(0, len(texts), batch_size):
            batch = texts[i:i + batch_size]
            tasks.append(embed_batch(batch))

        results = await asyncio.gather(*tasks)

        # 결과 병합
        embeddings = []
        for batch_result in results:
            embeddings.extend(batch_result)

        return embeddings


# 사용
async def main():
    embedder = AsyncEmbedder()
    embeddings = await embedder.embed_async(["텍스트1", "텍스트2", ...])
    print(f"생성된 임베딩: {len(embeddings)}개")

asyncio.run(main())
```

12.3 벡터 검색 최적화

```
class OptimizedVectorSearch:
    """최적화된 벡터 검색"""

    def __init__(self, vector_store):
        self.vector_store = vector_store
        self.query_cache = {}

    def search_with_cache(self,
                          query: str,
                          query_embedding: list,
                          top_k: int = 10,
                          cache_ttl: int = 3600) -> list:
        """쿼리 캐싱"""
```

```
        import time

        cache_key = hashlib.md5(query.encode()).hexdigest()

        if cache_key in self.query_cache:
            cached, timestamp = self.query_cache[cache_key]
            if time.time() - timestamp < cache_ttl:
                return cached

        results = self.vector_store.search(query_embedding, top_k)
        self.query_cache[cache_key] = (results, time.time())

        return results

    def batch_search(self,
                     queries: List[str],
                     query_embeddings: List[list],
                     top_k: int = 10) -> List[list]:
        """배치 검색"""
        # Pinecone은 네이티브 배치 지원
        if hasattr(self.vector_store, 'batch_query'):
            return self.vector_store.batch_query(query_embeddings, top_k)

        # 그 외: 병렬 처리
        from concurrent.futures import ThreadPoolExecutor

        with ThreadPoolExecutor(max_workers=10) as executor:
            results = list(executor.map(
                lambda emb: self.vector_store.search(emb, top_k),
                query_embeddings
            ))

        return results
```

---

13. 트러블슈팅

13.1 일반적인 문제

문제	원인	해결책
검색 결과 품질 낮음	청크 크기 부적절	청크 크기 조정 (250-500 토큰)
관련 문서 누락	Top-K 너무 작음	Top-K 증가 (20 이상)
응답이 사실과 다름	환각(Hallucination)	Reranker 적용, 프롬프트 개선
임베딩 속도 느림	배치 처리 미사용	배치 크기 최적화 (32-100)
메모리 부족	대용량 문서	스트리밍 처리, 청크 단위 처리
중복 결과	유사 청크 존재	MMR (Maximal Marginal Relevance) 적용

13.2 청킹 관련 문제

```
# 문제: 문맥이 끊어지는 청크
# 해결: 오버랩 증가 + Contextual Retrieval

def add_context_to_chunk(chunk: str, full_document: str) -> str:
    """청크에 문맥 추가 (Contextual Retrieval)"""
    prompt = f"""
    <document>
    {full_document[:2000]}...
    </document>

    <chunk>
    {chunk}
    </chunk>

    이 청크가 전체 문서에서 어떤 맥락에 있는지
    한 문장으로 요약하세요.
    """

    context = llm.generate(prompt)
    return f"{context}\n\n{chunk}"
```

13.3 검색 품질 문제

```
# 문제: 키워드 검색은 잘 되는데 의미 검색이 안 됨
# 해결: 하이브리드 검색 + 쿼리 확장

class QueryExpander:
    """쿼리 확장기"""

    def expand(self, query: str) -> List[str]:
        """쿼리를 여러 변형으로 확장"""
        prompt = f"""
        다음 검색 쿼리의 변형을 3개 생성하세요.
        동의어, 관련 용어, 다른 표현을 사용하세요.

        원본 쿼리: {query}

        변형 (줄바꿈으로 구분):
        """

        response = llm.generate(prompt)
        variants = [query] + response.strip().split("\n")

        return variants[:4]  # 원본 + 3개 변형


# 사용
expander = QueryExpander()
```

```
queries = expander.expand("RAG 시스템 구축 방법")
# ["RAG 시스템 구축 방법", "검색 증강 생성 구현", "RAG 파이프라인 설계", ...]

# 각 쿼리로 검색 후 결과 병합
all_results = []
for q in queries:
    results = pipeline.retrieve(q)
    all_results.extend(results)

# 중복 제거 및 재순위
unique_results = deduplicate_and_rerank(all_results)
```

13.4 환각(Hallucination) 방지

```
# 프롬프트 개선
ANTI_HALLUCINATION_PROMPT = """당신은 정직한 AI 어시스턴트입니다.

## 중요 규칙
1. 반드시 제공된 문서에 있는 정보만 사용하세요.
2. 문서에 없는 정보는 "문서에서 찾을 수 없습니다"라고 답하세요.
3. 추측하거나 일반 지식을 사용하지 마세요.
4. 답변에 사용한 문서 번호를 인용하세요. 예: [문서 1]
5. 불확실한 경우 "확실하지 않습니다"라고 표현하세요.

## 참고 문서
{contexts}

## 질문
{query}

## 답변 (문서 기반으로만 답변하세요)
"""

# 사후 검증
def verify_answer(answer: str, contexts: List[str]) -> dict:
    """답변 검증"""
    prompt = f"""
    답변이 다음 문서들에 기반했는지 검증하세요.

    문서들:
    {contexts}

    답변:
    {answer}

    JSON 형식으로 응답:
    {{
        "is_grounded": true/false,
        "unsupported_claims": ["지원되지 않는 주장들"],
        "confidence": 0.0-1.0
    }}
```

```
    """

    return json.loads(llm.generate(prompt))
```

---

14. 벤치마크 및 평가

14.1 RAG 평가 프레임워크

```
from dataclasses import dataclass
from typing import List, Dict

@dataclass
class RAGEvaluation:
    """RAG 평가 결과"""
    retrieval_metrics: Dict[str, float]
    generation_metrics: Dict[str, float]
    latency_metrics: Dict[str, float]

class RAGBenchmark:
    """RAG 벤치마크"""

    def __init__(self, pipeline: RAGPipeline, generator: RAGGenerator):
        self.pipeline = pipeline
        self.generator = generator

    def evaluate(self,
                 test_cases: List[dict],
                 ground_truth: List[dict]) -> RAGEvaluation:
        """
        전체 평가 실행

        test_cases: [{"query": "질문", "relevant_doc_ids": ["doc1", "doc2"]}]
        ground_truth: [{"query": "질문", "answer": "정답"}]
        """
        retrieval_metrics = self._evaluate_retrieval(test_cases)
        generation_metrics = self._evaluate_generation(test_cases, ground_truth)
        latency_metrics = self._evaluate_latency(test_cases)

        return RAGEvaluation(
            retrieval_metrics=retrieval_metrics,
            generation_metrics=generation_metrics,
            latency_metrics=latency_metrics
        )

    def _evaluate_retrieval(self, test_cases: List[dict]) -> Dict[str, float]:
        """검색 평가"""
        recalls = []
        precisions = []
        mrrs = []
```

```
        for case in test_cases:
            query = case["query"]
            relevant_ids = set(case["relevant_doc_ids"])

            results = self.pipeline.retrieve(query)
            retrieved_ids = {r["metadata"]["doc_id"] for r in results}

            # Recall
            hits = len(relevant_ids & retrieved_ids)
            recall = hits / len(relevant_ids) if relevant_ids else 0
            recalls.append(recall)

            # Precision
            precision = hits / len(retrieved_ids) if retrieved_ids else 0
            precisions.append(precision)

            # MRR
            for i, r in enumerate(results):
                if r["metadata"]["doc_id"] in relevant_ids:
                    mrrs.append(1 / (i + 1))
                    break
            else:
                mrrs.append(0)

        return {
            "recall@k": sum(recalls) / len(recalls),
            "precision@k": sum(precisions) / len(precisions),
            "mrr": sum(mrrs) / len(mrrs)
        }

    def _evaluate_generation(self,
                             test_cases: List[dict],
                             ground_truth: List[dict]) -> Dict[str, float]:
        """생성 평가"""
        from rouge_score import rouge_scorer

        scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'])

        rouge1_scores = []
        rouge2_scores = []
        rougeL_scores = []

        gt_map = {gt["query"]: gt["answer"] for gt in ground_truth}

        for case in test_cases:
            query = case["query"]
            contexts = self.pipeline.retrieve(query)
            response = self.generator.generate(query, contexts)

            predicted = response["answer"]
            reference = gt_map.get(query, "")

            if reference:
                scores = scorer.score(reference, predicted)
```

```
                rouge1_scores.append(scores['rouge1'].fmeasure)
                rouge2_scores.append(scores['rouge2'].fmeasure)
                rougeL_scores.append(scores['rougeL'].fmeasure)

        return {
            "rouge1": sum(rouge1_scores) / len(rouge1_scores) if rouge1_scores
else 0,
            "rouge2": sum(rouge2_scores) / len(rouge2_scores) if rouge2_scores
else 0,
            "rougeL": sum(rougeL_scores) / len(rougeL_scores) if rougeL_scores
else 0
        }

    def _evaluate_latency(self, test_cases: List[dict]) -> Dict[str, float]:
        """지연 시간 평가"""
        import time

        retrieval_times = []
        generation_times = []
        total_times = []

        for case in test_cases[:10]:  # 샘플링
            query = case["query"]

            # 검색 시간
            start = time.time()
            contexts = self.pipeline.retrieve(query)
            retrieval_time = time.time() - start
            retrieval_times.append(retrieval_time)

            # 생성 시간
            start = time.time()
            self.generator.generate(query, contexts)
            generation_time = time.time() - start
            generation_times.append(generation_time)

            total_times.append(retrieval_time + generation_time)

        return {
            "avg_retrieval_ms": sum(retrieval_times) / len(retrieval_times) *
1000,
            "avg_generation_ms": sum(generation_times) / len(generation_times) *
1000,
            "avg_total_ms": sum(total_times) / len(total_times) * 1000,
            "p95_total_ms": sorted(total_times)[int(len(total_times) * 0.95)] *
1000
        }


# 사용 예시
benchmark = RAGBenchmark(pipeline, generator)

test_cases = [
    {"query": "RAG란 무엇인가?", "relevant_doc_ids": ["doc1", "doc3"]},
```

```
    {"query": "청킹 전략은?", "relevant_doc_ids": ["doc2"]},
]

ground_truth = [
    {"query": "RAG란 무엇인가?", "answer": "RAG는 검색 증강 생성..."},
    {"query": "청킹 전략은?", "answer": "주요 청킹 전략으로는..."},
]

results = benchmark.evaluate(test_cases, ground_truth)
print(f"Recall@K: {results.retrieval_metrics['recall@k']:.3f}")
print(f"MRR: {results.retrieval_metrics['mrr']:.3f}")
print(f"ROUGE-L: {results.generation_metrics['rougeL']:.3f}")
print(f"평균 응답 시간: {results.latency_metrics['avg_total_ms']:.0f}ms")
```

14.2 A/B 테스트 프레임워크

```
class RAGABTest:
    """RAG A/B 테스트"""

    def __init__(self):
        self.results = {"A": [], "B": []}

    def compare_configs(self,
                        config_a: RAGConfig,
                        config_b: RAGConfig,
                        test_queries: List[str],
                        evaluator: callable) -> dict:
        """두 설정 비교"""
        pipeline_a = RAGPipeline(config_a)
        pipeline_b = RAGPipeline(config_b)

        for query in test_queries:
            # A 테스트
            results_a = pipeline_a.retrieve(query)
            score_a = evaluator(query, results_a)
            self.results["A"].append(score_a)

            # B 테스트
            results_b = pipeline_b.retrieve(query)
            score_b = evaluator(query, results_b)
            self.results["B"].append(score_b)

        return self._analyze_results()

    def _analyze_results(self) -> dict:
        """통계 분석"""
        import numpy as np
        from scipy import stats

        a_scores = np.array(self.results["A"])
        b_scores = np.array(self.results["B"])
```

```
        # t-검정
        t_stat, p_value = stats.ttest_rel(a_scores, b_scores)

        return {
            "config_a_mean": float(np.mean(a_scores)),
            "config_b_mean": float(np.mean(b_scores)),
            "improvement": float((np.mean(b_scores) - np.mean(a_scores)) /
np.mean(a_scores) * 100),
            "p_value": float(p_value),
            "significant": p_value < 0.05,
            "winner": "B" if np.mean(b_scores) > np.mean(a_scores) and p_value <
0.05 else "A" if p_value < 0.05 else "No significant difference"
        }


# 사용 예시
ab_test = RAGABTest()

config_a = RAGConfig(chunk_size=500, use_reranker=False)
config_b = RAGConfig(chunk_size=500, use_reranker=True)

results = ab_test.compare_configs(
    config_a, config_b,
    test_queries=["질문1", "질문2", ...],
    evaluator=lambda q, r: calculate_relevance_score(q, r)
)

print(f"Config A 평균: {results['config_a_mean']:.3f}")
print(f"Config B 평균: {results['config_b_mean']:.3f}")
print(f"개선율: {results['improvement']:.1f}%")
print(f"통계적 유의성: {results['significant']}")
print(f"승자: {results['winner']}")
```

---

15. 프로덕션 배포 가이드

15.1 아키텍처 설계

```
┌──────────────────────────────────────────────────────────────────┐
│                    Production RAG Architecture                   │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  [Client]                                                        │
│      │                                                           │
│      ▼                                                           │
│  ┌─────────┐    ┌─────────┐    ┌─────────┐                      │
│  │   LB    │ → │   API   │ → │  Cache  │                      │
│  │(Nginx)  │    │(FastAPI)│    │ (Redis) │                      │
│  └─────────┘    └────┬────┘    └────┬────┘                      │
│                      │              │                            │
```

```
│          ┌──────────┴──────────┴──────────┐                │
│          │          │          │          │                │
│          ▼          ▼          ▼          ▼                │
│   ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐      │
│   │ Embedding│ │  Vector  │ │   LLM    │ │ Reranker │      │
│   │  Service │ │    DB    │ │ Service  │ │ Service  │      │
│   └──────────┘ └──────────┘ └──────────┘ └──────────┘      │
│       │          │          │          │                   │
│       └──────────┴──────────┴──────────┘                   │
│                          │                                 │
│                          ▼                                 │
│                   ┌──────────┐                             │
│                   │  Queue   │  (Background Jobs)          │
│                   │ (Celery) │  - Document Ingestion       │
│                   └──────────┘  - Embedding Updates        │
│                                                            │
└────────────────────────────────────────────────────────────┘
```

15.2 FastAPI 서버 구현

```python
from fastapi import FastAPI, HTTPException, BackgroundTasks
from pydantic import BaseModel
from typing import List, Optional
import redis
import json

app = FastAPI(title="RAG API", version="1.0.0")

# 캐시
cache = redis.Redis(host='localhost', port=6379, db=0)
CACHE_TTL = 3600  # 1시간

# 파이프라인 초기화
pipeline = RAGPipeline(RAGConfig())
generator = RAGGenerator()

class QueryRequest(BaseModel):
    query: str
    top_k: Optional[int] = 10
    filter: Optional[dict] = None
    use_cache: Optional[bool] = True

class QueryResponse(BaseModel):
    answer: str
    contexts: List[dict]
    latency_ms: float

class IngestRequest(BaseModel):
    documents: List[dict]

@app.post("/query", response_model=QueryResponse)
```

```python
async def query(request: QueryRequest):
    """RAG 쿼리 엔드포인트"""
    import time
    start = time.time()

    # 캐시 확인
    if request.use_cache:
        cache_key = f"rag:{hash(request.query)}"
        cached = cache.get(cache_key)
        if cached:
            return QueryResponse(**json.loads(cached))

    # 검색
    contexts = pipeline.retrieve(
        query=request.query,
        top_k=request.top_k,
        filter=request.filter
    )

    # 생성
    response = generator.generate(request.query, contexts)

    latency = (time.time() - start) * 1000

    result = QueryResponse(
        answer=response["answer"],
        contexts=contexts,
        latency_ms=latency
    )

    # 캐시 저장
    if request.use_cache:
        cache.setex(cache_key, CACHE_TTL, json.dumps(result.dict()))

    return result

@app.post("/ingest")
async def ingest(request: IngestRequest, background_tasks: BackgroundTasks):
    """문서 인제스트 (백그라운드)"""
    background_tasks.add_task(pipeline.ingest, request.documents)
    return {"status": "accepted", "documents": len(request.documents)}

@app.get("/health")
async def health():
    """헬스 체크"""
    return {"status": "healthy"}

# 실행: uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

15.3 모니터링

```python
from prometheus_client import Counter, Histogram, start_http_server
import logging

# 메트릭
QUERY_COUNT = Counter('rag_queries_total', 'Total queries')
QUERY_LATENCY = Histogram('rag_query_latency_seconds', 'Query latency')
RETRIEVAL_HITS = Counter('rag_retrieval_hits', 'Retrieval cache hits')
ERROR_COUNT = Counter('rag_errors_total', 'Total errors', ['type'])

# 로깅
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger('rag')

class MonitoredRAGPipeline(RAGPipeline):
    """모니터링이 추가된 RAG 파이프라인"""

    def retrieve(self, query: str, **kwargs):
        QUERY_COUNT.inc()

        with QUERY_LATENCY.time():
            try:
                results = super().retrieve(query, **kwargs)
                logger.info(f"Query: {query[:50]}... | Results: {len(results)}")
                return results
            except Exception as e:
                ERROR_COUNT.labels(type=type(e).__name__).inc()
                logger.error(f"Query failed: {query[:50]}... | Error: {e}")
                raise

# Prometheus 메트릭 서버 시작
start_http_server(9090)
```

---

16. 참고 자료

16.1 공식 문서

- LangChain RAG
- LlamaIndex
- OpenAI Embeddings
- Anthropic Contextual Retrieval

16.2 논문

- RAG: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
- Lost in the Middle
- Precise Zero-Shot Dense Retrieval without Relevance Labels

16.3 도구

- Ragas - RAG 평가 프레임워크
- Chonkie - 청킹 라이브러리
- Docling - 문서 파싱